mini-redis 8·9단계 발표 대본

Nov 14, 2026 · @Velocity

쓰는 법과 시간 배분

발표는 약 10분, 슬라이드 27장입니다. 장 번호는 발표 자료의 순서와 같고, 굵은 글씨는 꼭 말할 문장입니다. 쇼핑몰 화면 장(5~8장, 17~20장)은 자동으로 재생되니 화면을 가리키며 말하면 됩니다.

구간	장	시간
앞부분	1~4장	1분
1부 · mini-redis 없을 때	5~9장	2분
2부 · 코드	10~15장	2분 30초
3부 · mini-redis 있을 때	16~21장	2분
검증	22~24장	1분 30초
마무리	25~27장	1분

발표 전에 준비할 것:

- [ ] 받은 파일을 모두 덮어쓰고 `python -m pytest -q` 가 45 passed인지 확인
- [ ] `python -m bench.benchmark --markdown` 으로 다시 재서 22장 숫자 바꾸기
- [ ] 질문이 나오면 쓸 터미널 하나 열어 두기 (`python -m demo.scenario --step --delay 1`)
- [ ] 발표 자료를 팀에 공유 (공유 메뉴)

앞부분 (1~4장, 1분)

1장 · 표지

안녕하세요, 8단계와 9단계를 맡은 김은기입니다. **저는 앞에서 만든 mini-redis를 실제 쇼핑몰 앱에 캐시로 붙여 보고, 효과를 재는 일을 했습니다.**

2장 · 맡은 일과 결과

8단계는 데모 API입니다. 같은 걸 두 번 조회하면 두 번째부터 캐시에서 나와 3초가 4ms가 되고, 가격을 바꾸면 캐시를 지워 새 가격이 나옵니다. 9단계는 성능 비교로, DB에 가는 횟수가 다섯 번에서 한 번으로 줄었습니다. 여기에 서버 두 대 비교와 테스트 14개를 더 만들었습니다.

3장 · 과제 요구사항

과제 요구사항 열 줄을 하나씩 대 봤습니다. 파란 글씨가 제 부분입니다. **무효화, API 응답 저장, 성능 비교, 기능 테스트가 8·9단계로 채워졌고, 선택 항목인 데이터 유지만 남았습니다.**

4장 · 폴더 구조

소망님의 mini_redis 폴더는 한 줄도 건드리지 않고, 그걸 쓰는 앱을 demo 폴더에 따로 만들었습니다. 이유는 세 가지입니다. 저장소와 앱은 실제로 다른 프로그램이라서고, cache.py에 두 종류를 두면 캐시만 바꿔 끼워 비교할 수 있어서고, DB를 SQLite 파일로 두면 서버를 두 대 띄워도 원본이 하나여서입니다.

1부 · mini-redis 없을 때 (5~9장, 2분)

5장 · 1부 표지

먼저 mini-redis가 없을 때를 보겠습니다. 캐시를 각 서버 안에 두는, 가장 쉬운 방법입니다. 서버 한 대일 땐 문제가 없는데, 3차 체크인에서 코치님이 짚으신 것처럼 **서버가 여러 대가 되면 문제가 생깁니다.** 왼쪽이 서버 A, 오른쪽이 서버 B, 아래는 데이터가 실제로 저장된 곳입니다.

6장 · 장면 ④ 같은 데이터를 서버 수만큼 저장

서버 A에서 처음 열면 DB에 가서 3초 걸리고, A의 캐시에 넣습니다. 다음 고객이 서버 B로 가면, B는 A의 캐시를 모르니까 또 DB에 갑니다. 아래에 같은 내용이 두 곳에 생기고 DB 조회가 2번이 됩니다. **가격이 틀린 건 아니지만, 서버가 10대면 DB에 10번 가는 낭비입니다.**

7장 · 장면 ① 가격이 왔다 갔다

관리자가 서버 A에서 4만 원으로 할인합니다. A는 자기 캐시를 지우지만, 아래를 보시면 B의 캐시에는 5만 원이 남아 있습니다. 그래서 B로 간 고객은 빨간 「옛 가격」 5만 원을 봅니다. **새로고침할 때마다 4만 원과 5만 원이 번갈아 보이고, B의 캐시가 만료될 때까지 최대 60초 동안 이렇게 됩니다.**

8장 · 장면 ③ 배포하면 DB가 몰림

서버 A를 재시작합니다. 로컬 캐시는 프로그램 메모리 안의 dict라서 프로그램이 꺼지면 함께 사라집니다. 재시작 직후 고객은 다시 DB에서 3초를 기다립니다. **서버 수십 대를 한꺼번에 배포하면 모든 요청이 DB로 몰립니다.**

9장 · 장면 ② 로그인이 풀림

서버 A에서 로그인하면 세션이 A 안에만 저장됩니다. 다음 페이지 요청이 B로 가면 B는 이 세션을 모르니까 「로그인이 필요합니다」가 뜹니다. **고객 입장에서는 멀쩡히 쓰다가 갑자기 로그아웃되는 겁니다.**

2부 · 무엇을 어떻게 고쳤나 (10~15장, 2분 30초)

10장 · 2부 표지

네 장면의 원인은 하나입니다. **캐시가 각 서버 안에 있다는 것.** 그래서 캐시를 서버 밖 mini-redis로 옮겼고, 코드에서 바뀐 곳은 네 군데입니다.

11장 · 코드 1, 캐시 바꿔 끼우기

LocalCache는 1부에서 본 방식으로, 이 서버 메모리 안의 dict입니다. RedisCache는 서버 밖 mini-redis에 HTTP로 물어봅니다. 둘 다 get, set, delete라는 같은 함수를 갖고 있어서, **앱은 어느 쪽을 쓰는지 모르고, 한 줄만 바꾸면 1부가 되고 3부가 됩니다.** 그래서 캐시 위치 말고는 모두 같은 공정한 비교입니다.

12장 · 코드 2, 조회

캐시에 있으면 바로 돌려주고 4ms입니다. 없으면 DB에서 3초 걸려 가져오고, 다음 사람을 위해 60초짜리로 캐시에 넣습니다. 이 방식을 Cache-Aside라고 합니다. **무엇을 캐시할지는 mini-redis가 아니라 앱이 정합니다.**

13장 · 코드 3, 가격 수정 (무효화)

DB를 고치고 캐시를 지웁니다, 두 줄입니다. 「앱이 지운다」는 앱이 mini-redis에 「이 키 지워 줘」라는 HTTP 요청을 보낸다는 뜻입니다. mini-redis는 DB가 바뀐 걸 모르기 때문에, **DB와 캐시를 둘 다 아는 앱이 지워야 합니다.** 순서도 중요합니다. 캐시를 먼저 지우면, DB를 고치기 전 틈에 다른 고객이 옛 가격을 다시 캐시에 넣을 수 있습니다.

14장 · 코드 4, 로그인

로그인하면 세션을 30분짜리로 캐시에 저장하고, 다른 페이지에서는 그 키가 있는지 봅니다. **세션을 위해 새로 만든 기능은 없습니다. 인기 상품과 같은 set과 get을 그대로 썼습니다.**

15장 · TTL은 왜 60초인가

TTL은 무효화를 놓쳤을 때의 안전망입니다. 기능을 추가하면서 delete를 빼먹거나, mini-redis가 잠깐 꺼져 삭제가 실패하거나, DB를 직접 고치는 경우죠. **그래서 「놓쳤을 때 틀린 값을 얼마나 참을 수 있나」로 정했습니다.** 5초면 1분에 DB를 12번 가서 캐시 의미가 없고, 10분이면 놓쳤을 때 10분간 틀린 가격이 보입니다. 인기 상품 목록은 초 단위로 바뀌지 않아 60초를 골랐습니다.

3부 · mini-redis 있을 때 (16~21장, 2분)

16장 · 3부 표지

같은 네 장면을 mini-redis로 봅니다. 코드는 방금 본 그대로고 캐시만 바꿔 끼웠습니다. **화면 아래를 보시면 이번엔 캐시 상자가 하나입니다.**

17장 · ④ 같은 데이터는 한 곳에만

A가 DB에서 가져와 mini-redis에 넣으면, B는 그걸 바로 씁니다. **DB 조회는 1번에서 멈춥니다.**

18장 · ① 키 하나만 지우면 모두 새 가격

A에서 할인하면 mini-redis의 키를 지웁니다. 캐시가 하나뿐이니 이 한 번으로 끝이고, **B로 가도 4만 원입니다. 옛 가격이 남을 곳 자체가 없습니다.**

19장 · ③ 재시작해도 캐시는 그대로

서버 A를 재시작해도 아래 mini-redis 상자는 그대로입니다. 캐시가 다른 프로그램 안에 있으니까요. **재시작 직후에도 바로 응답하고 DB 조회는 0번입니다.** 다만 mini-redis 자신을 재시작하면 사라지고, 그걸 막는 게 14단계입니다.

20장 · ② 어느 서버로 가도 로그인 유지

세션이 mini-redis에 있으니 B도 찾습니다. **새로 만든 기능 없이 같은 set과 get입니다.**

21장 · 결과 비교

한 표로 모으면 이렇습니다. **같은 코드, 같은 순서인데 캐시 위치만 바꿈더니 네 장면이 전부 달라졌습니다.** 앞의 화면은 이해를 돕는 재현이고, 실제 코드로는 아래 명령으로 서버를 두 대씩 띄워 같은 결과를 확인했습니다.

검증 (22~24장, 1분 30초)

22장 · 9단계 성능 비교

캐시에 있으면 3초가 4ms, 대략 700배입니다. 다만 DB 3초는 제가 일부러 만든 값이라, **배수보다 DB에 다섯 번 갈 걸 한 번만 갔다는 게 더 중요합니다.** 같은 주소에 nocache=1만 붙여 캐시를 건너뛴 것과 비교했고, 첫 조회는 원래 느리니 따로 재었습니다.

23장 · 테스트

테스트 14개를 다섯 묶음으로 만들었습니다. 완료 기준 묶음은 8단계 완료 기준을 자동으로 확인하고, **서버 두 대 묶음은 화면으로 본 1부와 3부의 차이를 실제 코드로 증명합니다.** 캐시 고장 묶음은 mini-redis가 꺼져도 서비스가 DB로 계속 답하는지, 만료 묶음은 TTL 안전망이 정말 일하는지 봅니다. 그리고 **캐시 삭제를 일부러 빼고 돌려 봤더니 세 개가 실패했습니다.** 무효화가 빠지면 테스트가 잡아낸다는 뜻입니다.

24장 · 트러블슈팅, 시연에서만 TTL 600초

시연을 자동으로 돌리면 잘 됐는데, 발표 연습으로 한 단계씩 넘기니 서버 B가 옛 가격이 아니라 새 가격을 보여 줬습니다. 재시작 탓도, 서버끼리 삭제가 전달된 것도 아니었고, **다른 건 걸린 시간뿐이었습니다. 설명하는 사이 60초가 지나 캐시가 만료된 겁니다.** 그래서 기본값 60초는 두고 시연 스크립트만 600초를 주게 했고, TTL 2초면 깨지고 600초면 재현되는 걸 확인했습니다. 덕분에 「TTL을 짧게 하면 되지 않냐」에 답할 수 있게 됐습니다. 로컬 캐시도 결국 새 값을 가져오지만, 그 사이엔 틀린 가격이 보입니다.

마무리 (25~27장, 1분)

25장 · 다섯 문제 비교

로컬 캐시의 다섯 문제를 레디스가 무엇으로 풀었는지입니다. 넷은 오늘 시연으로 보여 드렸고, ⑤만 남았습니다. **다섯 문제의 공통 해결은 「상태를 서버 밖 한 곳으로」이고, 그 한 곳에서 키로 값을 찾는 게 우리가 만든 해시 테이블입니다.**

26장 · 세부 기능 비교

레디스와 다르게 한 곳마다 이유가 있습니다. 예를 들어 레디스는 싱글 스레드인데 우리는 lock을 씁니다. Flask가 요청을 여러 스레드로 받기 때문이고, **lock으로 같은 효과를 냅니다.** 통신을 HTTP로 한 건 과제의 「외부에서 쉽게 사용」 때문입니다.

27장 · 남은 것과 정리

지금 못 보여 드린 건 전부 mini-redis 자체에 기능이 없어서입니다. 12단계 증가 명령, 13단계 메모리 관리, 14단계 파일 저장입니다. **세 가지 모두 mini_redis 폴더를 고치는 일이라 팀과 정해야 하고, 한다면 12단계부터 제안드립니다.** 그리고 TTL 60초와 캐시 키 이름은 분담안에 제안으로 적힌 값이라 확정해 주시면 좋겠습니다. 감사합니다.

예상 질문과 짧은 답

질문	답
TTL을 짧게 하면 ①이 풀리지 않나요?	틀린 가격이 보이는 시간만 짧아지고 없어지지는 않습니다. 대신 DB 조회가 늘어납니다. 진짜 해결은 가격이 바뀌는 순간 지우는 것입니다.
mini-redis가 죽으면 서비스가 멈추나요?	아닙니다. 통신 실패를 「캐시에 없음」으로 처리해 DB로 답합니다. 느려질 뿐이고, 테스트로도 확인했습니다.
레디스가 캐시를 알아서 채워 주나요?	아닙니다. 무엇을 넣고 언제 지울지는 앱이 정합니다. 레디스가 알아서 하는 건 TTL이 지난 키를 지우는 것뿐입니다.
왜 덮어쓰지 않고 지우나요?	수정이 동시에 오면 DB와 캐시의 쓰기 순서가 엇갈려 어긋날 수 있습니다. 지우면 순서가 바뀌어도 결과가 「캐시 없음」으로 같습니다.
lock이 있는데 왜 ⑤는 안 풀리나요?	lock은 명령 하나씩만 안전하게 합니다. 재고 차감은 get → 빼기 → set으로 명령이 두 번이라 그 사이에 끼어들 수 있습니다. 12단계 증가 명령이 필요합니다.
왜 HTTP인가요? 레디스는 RESP인데	과제가 「외부에서 쉽게 사용할 수 있는 API」를 요구해서입니다. curl로 바로 확인할 수 있고, 대신 히트가 1ms가 아니라 4ms입니다. 1단계에서 팀이 정한 것입니다.
시연 화면은 진짜 코드인가요?	우리 코드와 같은 로직을 재현한 화면입니다. 실제 코드로는 demo.scenario가 서버를 두 대씩 띄워 같은 결과를 내고, 테스트 14개가 같은 장면을 확인합니다.